Middle / B range 55/60

Coursework of Smart Grid Technologies

Shikun Xu CID: 01248192 Group: Msc - Future Power Networks

Part 1 – Analytical Calculations

1. Compute the availability of each component

$$Availability = \frac{MTTF}{MTBF}$$

$$MTBF = MTTF + MTTR$$

Since the component failure processes are constant-rate processes, we can compute $MTTF$ as the reciprocal of λ.

$$MTTF = \frac{1}{\lambda} = \frac{1}{Failure\ rate \times Cable\ length}$$

We use these three formulas to calculate Availability of each component and summarize the results into the table below.

Component	MTTF	MTTR	MTBF	Availability
T1	100 years	10 days	36510 days	0.999726
T2	100 years	10 days	36510 days	0.999726
F1	10 years	2 days	3652 days	0.999452
F2	6.67 years	2 days	2435.33 days	0.999179
F3	1.67 years	10 hours	14610 hours	0.999316

ok

2. Compute the loss of load probability (LOLP) for this network. Consider whether a single transformer is sufficient to supply all loads. You may want to use reliability block diagrams.

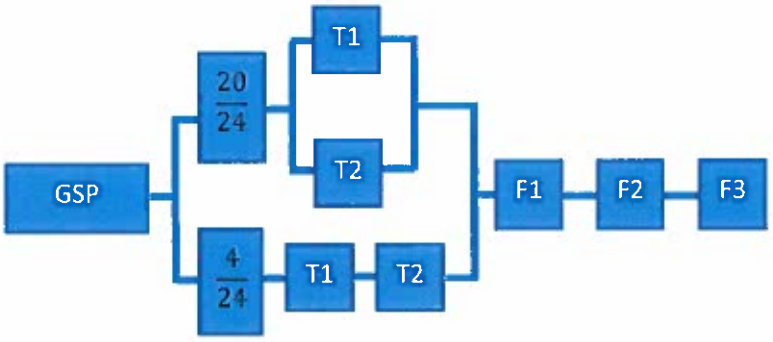
If we want one transformer to supply all loads , due to the thermal rating of each transformer, we calculate the max *Power Consumption per Household*:

$$Power\ Consumption\ per\ Household_{max} = \frac{thermal\ rating}{number\ of\ all\ customers} = \frac{8{,}000{,}000}{11{,}000}$$

$$= 727\ \text{[W/household]} = 0.727\ \text{[kW/household]}$$

According to the daily load profile, we find that one transformer is enough to supply all loads for 20 hours of a day, while we need two transformers to supply all loads for the rest 4 hours.

Therefore, we can draw the reliability block diagram of the system.



$$r_{system} = \{\frac{20}{24}[1-(1-r_{T1})(1-r_{T2})] + \frac{4}{24}r_{T1}r_{T2}\}r_{F1}r_{F2}r_{F3} = 0.997857$$

$$\boxed{LOLP = 1 - r_{system} = 0.002143}$$

ok

3. **Compute the unavailability of power supplied to each load point (a,b,c).**

   Hint: first calculate the probability that two transformers are required to supply all load, which will depend on the hour of day and on whether other load points are being supplied.

   According to the fact that the transformer has a thermal rating of 8MVA and will trip when overloaded, we do not have to consider the real-time load managed situation. That means, if there is insufficient transformer capacity for real-time load, all load is disconnected.

   Hence, we can compute the maximum household power consumption for each feeder condition:

$$Power\ Consumption\ per\ Household_{max} = \frac{thermal\ rating}{number\ of\ customers\ connected}$$

We summarize the results into the table of load level as below:

Load Level (Number of Consumers connected)	Expression of Probability of Feeder Condition	Max Power Consumption with one Transformer (kW/household)	Disconnected Proportion of a Day
11000	$r_{F1}r_{F2}r_{F3}$	0.727	4/24
9000	$r_{F1}r_{F2}(1-r_{F3})$	0.889	0
6000	$(1-r_{F1})r_{F2}r_{F3}$	1.333	0
5000	$r_{F1}(1-r_{F2})$	1.6	0
4000	$(1-r_{F1})r_{F2}(1-r_{F3})$	2	0
0	$(1-r_{F1})(1-r_{F2})$	-	0

We compare the results of *Max Power Consumption with one Transformer* with the average household power consumption for each hour, we can find that only in the period of 4 hours from 18 to 21 under Load Level of 11,000 consumers, one transformer can not supply all load.

According to the results in the table above, we can calculate the Probability that two transformers are required to supply all load:

$$P(\text{two transformers are required to supply all load}) = r_{F1}r_{F2}r_{F3} \times \frac{4}{24} = 0.166325$$

$$P(\text{one transformer is enough to supply all load}) = 1 - r_{F1}r_{F2}r_{F3} \times \frac{4}{24}$$
$$= 0.833675 \ (contain\ the\ probability\ of\ Load\ Level = 0)$$

Then, we consider two situations respectively:

(1) Two transformers are required to supply all load (denote this situation as $\mathbf{T_2}$)

Load Level (Number of Consumers)	Expression of Probability of Feeder Condition	Probability of Feeder Condition	Proportion of a Day
11000	$r_{F1}r_{F2}r_{F3}$	0.997948	4/24
Total Probability	0.166325		

Under this situation,

$$P_{T2}(load\ point\ a\ is\ supplied) = r_{T1}r_{T2} \times 0.166325 = 0.166234$$

$$P_{T2}(load\ point\ b\ is\ supplied) = r_{T1}r_{T2} \times 0.166325 = 0.166234$$

$$P_{T2}(load\ point\ c\ is\ supplied) = r_{T1}r_{T2} \times 0.166325 = 0.166234$$

(2) One transformer is enough to supply all load (denote this situation as T_1)

Load Level (Number of Consumers)	Expression of Probability of Feeder Condition	Probability of Feeder Condition	Proportion of a Day
11000	$r_{F1}r_{F2}r_{F3}$	0.997948	20/24
9000	$r_{F1}r_{F2}(1-r_{F3})$	0.000683	1
6000	$(1-r_{F1})r_{F2}r_{F3}$	0.000547	1
5000	$r_{F1}(1-r_{F2})$	0.000821	1
4000	$(1-r_{F1})r_{F2}(1-r_{F3})$	0	1
0	$(1-r_{F1})(1-r_{F2})$	0	1
Total Probability	0.833675		

Under this situation,

$$P_{T1}(load\ point\ a\ is\ supplied) = [1-(1-r_{T1})(1-r_{T2})]\times(0.997948\times\frac{5}{6}+0.000683+0.000821) = 0.833127$$

$$P_{T1}(load\ point\ b\ is\ supplied) = [1-(1-r_{T1})(1-r_{T2})]\times(0.997948\times\frac{5}{6}+0.000683+0.000547) = 0.832853$$

$$P_{T1}(load\ point\ c\ is\ supplied) = [1-(1-r_{T1})(1-r_{T2})]\times(0.997948\times\frac{5}{6}+0.000547) = 0.832170$$

Therefore,

$$Unavailability_a = 1-[P_{T2}(load\ point\ a\ is\ supplied)+P_{T1}(load\ point\ a\ is\ supplied)] = 1-(0.166234+0.833127) = 0.000639$$

$$Unavailability_b = 1-[P_{T2}(load\ point\ b\ is\ supplied)+P_{T1}(load\ point\ b\ is\ supplied)] = 1-(0.166234+0.832853) = 0.000913$$

$$Unavailability_c = 1-[P_{T2}(load\ point\ c\ is\ supplied)+P_{T1}(load\ point\ c\ is\ supplied)] = 1-(0.166234+0.832170) = 0.001596$$

ok

4. **Using the computed availabilities, determine the CML for this network.**

Since all faults are assumed to be persistent and last more than 3 minutes, all the incidents should be taken into account to compute CML.

Therefore, we compute the disconnected minutes for each load.

$$T_a = Unavailability_a \times 365 \times 24 \times 60 = 335.86 \text{ [mins]}$$

$$T_b = Unavailability_b \times 365 \times 24 \times 60 = 479.87 \text{ [mins]}$$

$$T_c = Unavailability_c \times 365 \times 24 \times 60 = 838.86 \text{ [mins]}$$

$$\boxed{CML = \frac{T_a \times 5000 + T_b \times 4000 + T_c \times 2000}{11000} = 479.68 \text{ [mins]}}$$

ok

5. **Would the CML index for this network be considered acceptable for real distribution networks?**

As we learned from lectures, the CML in UK is around 80 minutes. Even we include exceptional events, the CML should be controlled less than 120 in UK. As we can see from Ofgem Electricity Distribution Annual Report for 2010-11, the target CMLs are always less than 100. Even if there may be different standards under other countries' contexts, the CML we get is really too large. Therefore, we must take actions to enhance the reliability of the case distribution system.

ok

Bonus

Compute the EENS for this network

For comparison with latter simulations, I also compute the analytical EENS for this network. I use T_i to denote r_{T_i} , F_i to denote r_{F_i}, $\neg T_i$ to denote $(1 - r_{T_i})$, $\neg F_i$ to denote $(1 - r_{F_i})$, M_{all} to denote mean household power consumption for all time slots and M_{peak} to denote mean household power consumption for peak periods (hour at 18, 19, 20, 21 of a day).

1

$$EENS = [(\neg F_1)M_{all} + F_1(\neg T_1)(\neg T_2)M_{all} + \frac{1}{6}F_1F_2F_3 \times 2T_1(\neg T_2)M_{peak}] \times 5000 \times 8760 +$$

$$[(\neg F_2)M_{all} + F_2(\neg T_1)(\neg T_2)M_{all} + \frac{1}{6}F_1F_2F_3 \times 2T_1(\neg T_2)M_{peak}] \times 4000 \times 8760 +$$

$$[(\neg F_3)M_{all} + F_3(\neg F_2)M_{all} + F_2F_3(\neg T_1)(\neg T_2)M_{all} + \frac{1}{6}F_1F_2F_3 \times 2T_1(\neg T_2)M_{peak}] \times 2000 \times 8760 = 45.88 \text{ [MWh]}$$

Summary of Comments on Shikun Xu.pdf

Page: 5

Number: 1 Author: Simon Tindemans Date: Indeterminate

Simon Tindemans: very nice!

Part 2 – State Sampling Monte Carlo

6. **Define what constitutes a 'state' for this problem, and implement a function that samples a network state:**

   a. Sample a random state according to the availabilities computed in (1) and the load model defined above. Hint: you can use the Matlab command `randi(.)` to generate a random hour-of-day.

   b. Return the sampled state.

   First, we sample a random state of the components according to their availabilities. Then, we sample a random time and determine the power consumption per household at the time according to the load profile. After that, we samples the load according to the status of the feeders.

   The system state is defined by the status of the transformers (T1, T2) and the feeders (F1, F2, F3) and the power consumption per household (load_household) and load level (load). The sampled state can be returned as a vector (T1, T2, F1, F2, F3, load_household, load), where the first five components are binary and the last two are real numbers.

   Matlab code for the function **sample1()** is appended at the end of the document.

   ok

7. **Write a function takes a network state as an input and computes three impact values, which will be used to compute the LOLP, expected-CML and EENS.**

   In this system, since we assume that the transformer will trip when overloaded, the problem is much simplified. There are two cases, one scenario is the available transformer capacity can supply all the load connected by the feeders and the other one is not. If available transformer capacity can supply all the load connected by the feeders, then the load points status (OK or Failed) only depends on the status of the feeders. If the available transformer capacity can not supply all the load connected by the feeders, the transformer will trip and cause the blackout.

   Matlab code for the function **impact1(state)** is appended at the end of the document.

   ok

Page: 6

Number: 1 Author: Simon Tindemans Date: Indeterminate

Simon Tindemans: I would consider this part of the impact, because it incorporates the status of the feeders.

8. **Use the functions you created above to develop a state sampling Monte Carlo simulation that estimates the LOLP, expected CML and EENS. Use at least 100,000 samples and report the results accompanied by error estimates.**

   Matlab code for the script **run1.m** is appended at the end of the document.

   The results of a single experiment with **Sample Size = 100,000** are listed below.

   LOLP = 0.002200 ± 0.000148

   LOLP = [0.001910, 0.002490] (95% confidence)

   LOLP = 0.002200 ± 6.7 %

   expected-CML = 484.99 ± 35.89 minites

   expected-CML = [414.65, 555.32] minites (95% confidence)

   expected-CML = 484.99 minites ± 7.4 %

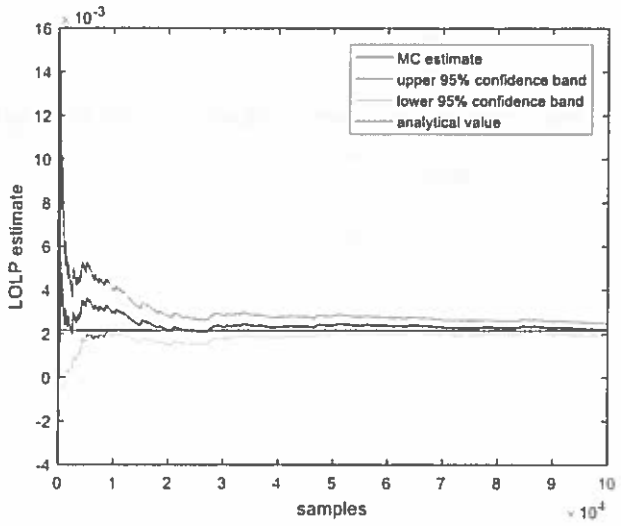
   EENS = 45.02 ± 3.75 MWh

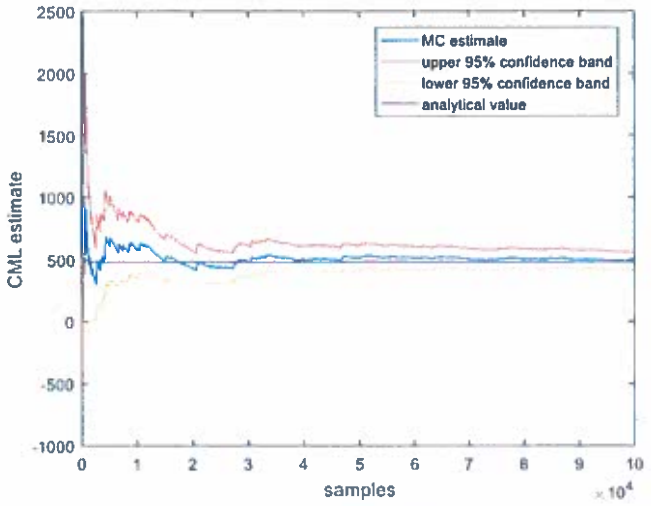
   EENS = [37.68, 52.37] MWh (95% confidence)

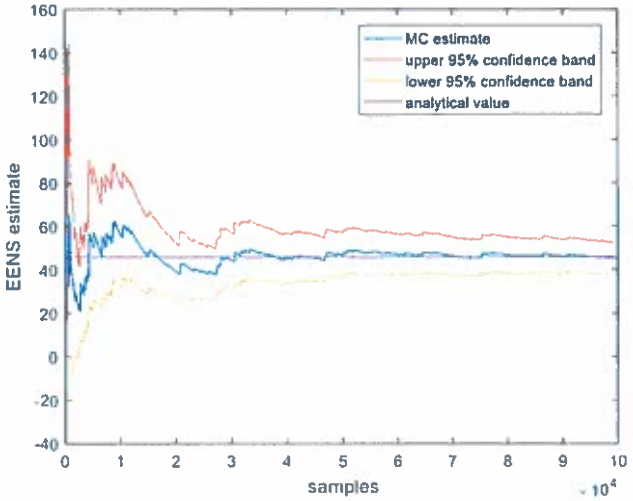
   EENS = 45.02 MWh ± 8.3 %

ok

We draw plots for the snapshot Monte Carlo estimates of LOLP, expected CML and EENS as a function of **sample size N**. For each figure, include a line for the analytical value. From the figures, we can confirm the tendency that if the **sample size N** turns to infinite, the expected value of the results will converge to the analytical values.







The results of a single experiment with **Sample Size = 2,000,000** are listed below

LOLP = 0.002135 ± 0.000033

LOLP = [0.002071, 0.002198] (95% confidence)

LOLP = 0.002135 ± 1.5 %

expected-CML = 484.67 ± 8.14 minites

expected-CML = [468.71, 500.64] minites (95% confidence)

expected-CML = 484.67 minites ± 1.7 %

EENS = 46.25 ± 0.89 MWh

EENS = [44.52, 47.99] MWh (95% confidence)

EENS = 46.25 MWh ± 1.9 %

9. Compare the LOLP and expected CML results to those obtained using analytical computations (question 2 and 4), and reflect on the relative advantages of each method.

100,000 samples:

Impact Metric	Analytical value	MC estimate	Error	95% confidence
LOLP	0.002143	0.002200	2.66%	[0.001910, 0.002490]
expected CML [minutes]	479.68	484.99	1.11%	[414.65, 555.32]
EENS [MWh]	45.88	45.02	1.87%	[37.68, 52.37]

2,000,000 samples:

Impact Metric	Analytical value	MC estimate	Error	95% confidence
LOLP	0.002143	0.002135	0.37%	[0.002071, 0.002198]
expected CML [minutes]	479.68	484.67	1.04%	[468.71, 500.64]
EENS [MWh]	45.88	46.25	0.81%	[44.52, 47.99]

We can see from the above tables that all the snapshot Monte Carlo estimate results are close to the analytical values. This confirms the correctness of simulation methodologies and codes. As we increase the sample size from 100,000 to 2,000,000, the errors reduce a lot and the 95% confidence bands shrink to much narrower intervals that are closer to the analytical values.

Relative advantages of Analytical method:

(1) It is an accurate method and the result is stable.

(2) Once the formula is determined, the result can be computed immediately and obviously the result is a precise answer.

Relative advantages of Snapshot Monte Carlo simulation:

(1) It is easy to implement. That means, we do not have to analyse the intrinsic attributes of the network and we only need to record the result of each sample state. In the case network, the microgrid has only five components, so it is relatively easy to compute analytical values. Even so, the expressions of expected-CML and EENS are very complicated. While, the real networks can be much more complex, computing analytical values of risk metrics will be unrealistic under real circumstances.

(2) It is a rapid way to deal with uncertainties and find their impacts. The snapshot experiments can be quickly simulated by computers. Sometimes we do not know some parameters or even the dimentionality of the system. Under these circumstances, we have no way to use analytical method and Monte Carlo state sample simulation is a convenient way to estimate the risk metrics of the system.

(3) The accuracy of the results can be predicted and controlled. If we need a more accurate result, we can sample more random states and the error will decrease in proportion to $\sqrt{N}$. In order to reduce the error by a factor k, we need k^2 times larger sample size.

ok

Number: 1 Author: Simon Tindemans Date: Indeterminate

Simon Tindemans: 'close to' requires quantification. In this case they are within the 95% confidence intervals, which gives confidence in both results.

Part 3 – TIME-SEQUENTIAL MONTE CARLO

10. Write a function that samples a random initial state of the system ($t = 0$). The state of the network elements should be sampled according to their availability, and the load can be initialised at hour 1.

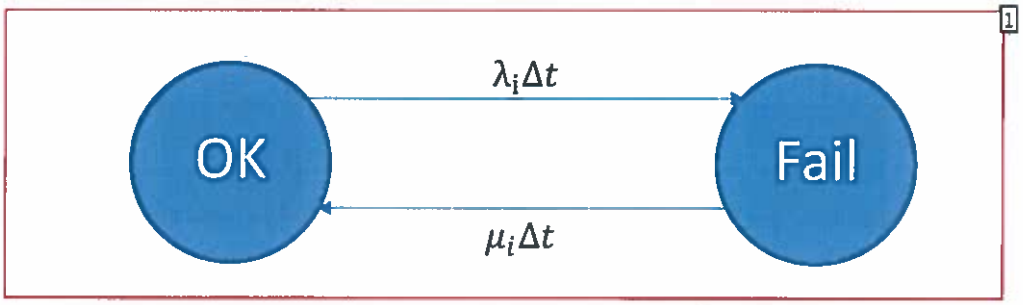
First, we sample a random state of the components according to their availabilities. Then, we initialize time at hour 1 and determine the power consumption per household at the time according to the load profile. After that, we samples the load according to the status of the feeders.

The system state is defined by the status of the transformers (T1, T2) and the feeders (F1, F2, F3) and the power consumption per household (load_household) and load level (load) and hour of a day (time). The sampled state can be returned as a vector (T1, T2, F1, F2, F3, load_household, load, time), where the first five components are binary and the last two are real numbers.

Matlab code for the function **sample1()** is appended at the end of the document.

ok

11. Determine the discrete time Markov chains for the transformers and feeders in the network, assuming a time step of $\Delta t = 1$ hour.



As the above figure shown, each component has two status: 'OK' and 'Failed' (i=T1,T2,F1,F2,F3). We calculate λ and μ in unit of [day^{-1}] and summarize the results into the table below.

Coponents	λ (day^{-1})	μ (day^{-1})
T1	0.0000274	0.1
T2	0.0000274	0.1
F1	0.0002740	0.5
F2	0.0004110	0.5
F3	0.0016438	2.4

Number: 1 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: the discrete time markov chain also has self-loops ok->ok and fail->fail.

In order to discrete set of states, we have to choose a small enough value of Δt to satisfy $\lambda \Delta t \leq 1$ and $\mu \Delta t \leq 1$ for each of the transition rates in the problem. On the other hand, small time steps slow down simulations. We have to compromise these two considerations and the given $\Delta t = 1$ hour is suitable for this simulation. In this case, we simulate a single year path with 8760 time steps. 1

Matlab code for the function **update_components(state)** is appended at the end of the document.

12. **Write a function that takes a network state at time t, evolves it to $t + 1$hour and returns the new state. Note that the load is not modelled as a Markov chain; the load level advances one hour at a time (i.e. exactly 24 hours for a full cycle).**

    Matlab code for the function **next_time_step(state)** is appended at the end of the document. ok

13. **Implement a time-sequential MC simulation that generates random paths of length T =1 year. For each year, it should compute the loss-of-load time, CML, CI and ENS. The values for all simulated years can be combined to estimate the LOLP, expected CML, EENS and expected CI. For the CI index, you can count every single instance customers are disconnected, irrespective of the cause.**

    In this system, since we assume that the transformer will trip when overloaded, the problem is much simplified. There are two cases, one scenario is the available transformer capacity can supply all the load connected by the feeders and the other one is not. If available transformer capacity can supply all the load connected by the feeders, then the load points status (OK or Failed) only depends on the status of the feeders. If the available transformer capacity can not supply all the load connected by the feeders, the transformer will trip and cause the blackout.

    We have to notice that for the CI index, we have to record the previous status of load points and always compare the current state with the previous state. Only the instances that the previous status of a load point is OK and the current statis of the load point is Failed should be taken into account to compute expected CI. In addition, we count every single instance customers are disconnected, irrespective of the cause.

    Matlab code for the function **year_path( )** is appended at the end of the document.

ok

Page: 11

Number: 1 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: what are the transition probabilities?

14. Sample at least 1,000 random years and report estimates (with error estimates) for LOLP, expected CML, EENS and expected CI. Compare the results with those obtained using the analytical method and state sampling simulations.

Matlab code for the script **run2.m** is appended at the end of the document.

The results of a single experiment with **Sample Size = 1000 years** are listed below.

LOLP = 0.002098 ± 0.000126

LOLP = [0.001851, 0.002344] (95% confidence)

LOLP = 0.002098 ± 6.0 %

expected-CML = 475.08 ± 36.32 minites

expected-CML = [403.90, 546.26] minites (95% confidence)

expected-CML = 475.08 minites ± 7.6 %

EENS = 46.28 ± 3.88 MWh

EENS = [38.67, 53.88] MWh (95% confidence)

EENS = 46.28 MWh ± 8.4 %

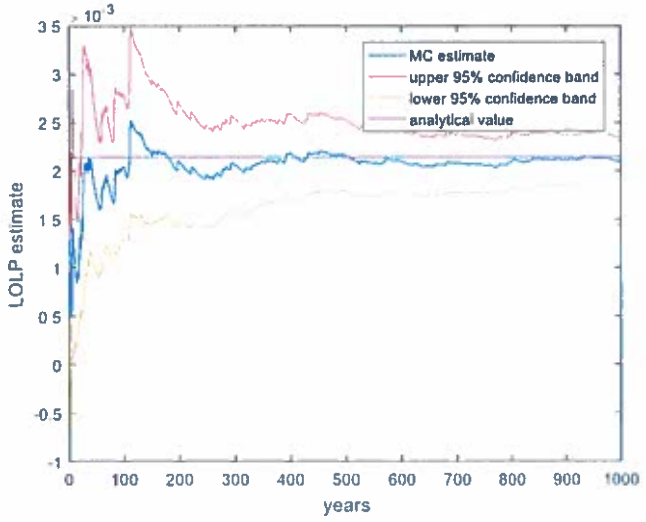
expected-CI = 47.65 ± 7.49 occurrences/year

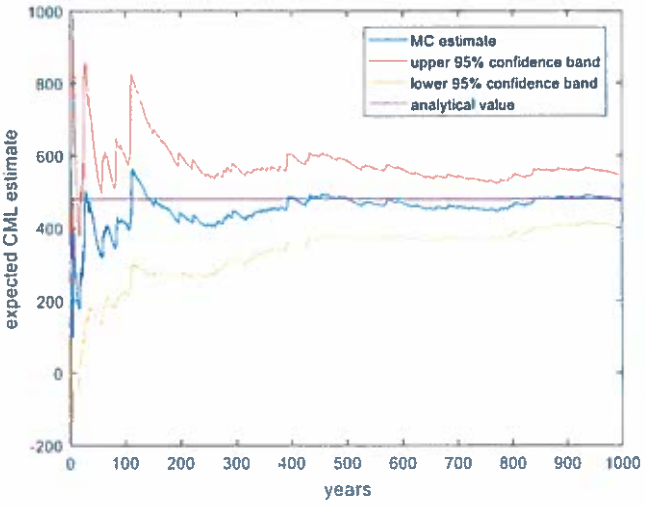
expected-CI = [32.97, 62.34] occurrences/year (95% confidence)

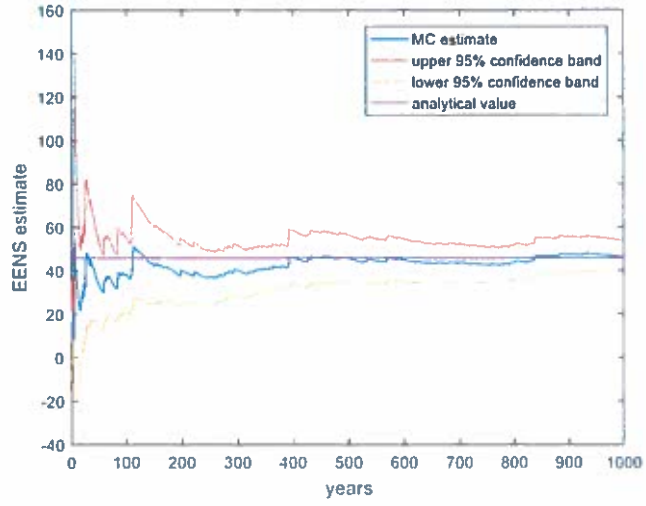
expected-CI = 47.65 occurrences/year ± 15.7 %

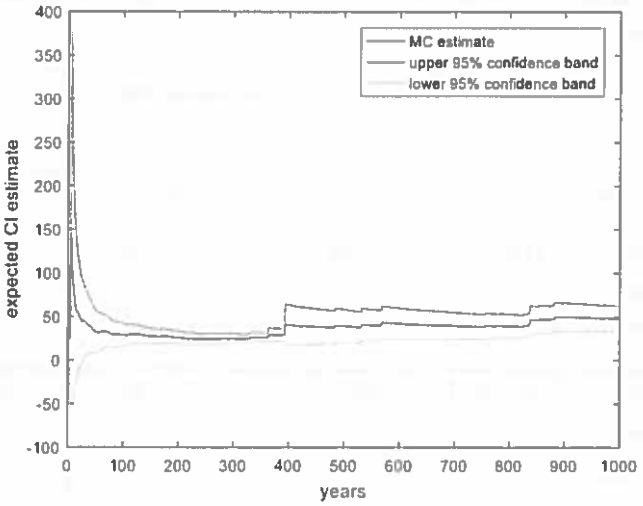
Impact Metric	Analytical value	MC estimate	Error	95% confidence
LOLP	0.002143	0.002098	2.10%	[0.001851, 0.002344]
expected CML [minutes]	479.68	475.08	0.96%	[403.90, 546.26]
EENS [MWh]	45.88	46.28	0.87%	[38.67, 53.88]
expected CI [occurrences]		47.65		[32.97, 62.34]

We draw plots for the Time-Sequential Monte Carlo estimates of LOLP, expected CML, EENS and expected CI as a function of **sample year size N**. For each figure, include a line for the analytical value (expected CI is excluded). From the figures, we can confirm the tendency that if the **sample year size N** turns to infinite, the expected value of the results will converge to the analytical values.


x 10^-3
MC estimate
upper 95% confidence band
lower 95% confidence band
analytical value
LOLP estimate
years


MC estimate
upper 95% confidence band
lower 95% confidence band
analytical value
expected CML estimate
years


MC estimate
upper 95% confidence band
lower 95% confidence band
analytical value
EENS estimate
years



The results of a single experiment with **Sample Size = 10,000 years** are listed below.

LOLP = 0.002177 ± 0.000040

LOLP = [0.002098, 0.002256] (95% confidence)

LOLP = 0.002177 ± 1.9 %

expected-CML = 485.62 ± 11.17 minites

expected-CML = [463.72, 507.52] minites (95% confidence)

expected-CML = 485.62 minites ± 2.3 %

EENS = 46.58 ± 1.16 MWh

EENS = [44.31, 48.85] MWh (95% confidence)

EENS = 46.58 MWh ± 2.5 %

expected-CI = 45.27 ± 2.06 occurrences/year

expected-CI = [41.24, 49.30] occurrences/year (95% confidence)

expected-CI = 45.27 occurrences/year ± 4.5 %

Impact Metric	Analytical value	MC estimate	Error	95% confidence
LOLP	0.002143	0.002177	1.59%	[0.002098, 0.002256]
expected CML [minutes]	479.68	485.62	1.24%	[463.72, 507.52]
EENS [MWh]	45.88	46.58	1.53%	[44.31, 48.85]
expected CI [occurrences]		45.27		[41.24, 49.30]

In order to compare the results derived from different methods, we summarize the values into the table below.

Impact Metric	LOLP	expected CML [minutes]	EENS [MWh]
Analytical value	0.002143	479.68	45.88
Snapshot MC estimate (N=100,000)	0.002200 ± 0.000148	484.99 ± 35.89	45.02 ± 3.75
Snapshot MC estimate (N=2,000,000)	0.002135 ± 0.000033	484.67 ± 8.14	46.25 ± 0.89
Time-sequential MC estimate (N=1,000 years)	0.002098 ± 0.000126	475.08 ± 36.32	46.28 ± 3.88
Time-sequential MC estimate (N=10,000 years)	0.002177 ± 0.000040	485.62 ± 11.17	46.58 ± 1.16

1

We can see from the above tables that all the results of snapshot Monte Carlo estimate and time-sequential Monte Carlo estimate are close to the analytical values. This confirms the correctness of simulation methodologies and codes.

As we increase the sample size, the errors reduce and the confidence bands shrink to much narrower intervals that are closer to the analytical values. Actually, the accuracy of the results can be predicted and controlled. If we need a more accurate result, we can sample more states and the error will decrease in proportion to $\sqrt{N}$. In order to reduce the error by a factor k, we need k^2 times larger sample size.

For snapshot Monte Carlo estimate, it is easier to program than time-sequential Monte Carlo estimate, since it can traverse the space rapidly and the samples are independent. However, we are not able to analyse system dynamics by this method, since the independent snapshots show no information about the dynamic properties of the system. Thus, we have to use other methods to program if we want to compute LOLF, Expected CI or SAIFI of the system. Also, if we want to get a good answer of the risk metrics rather than a barely reasonable answer, we have to increase the sample size a lot and spend much longer time.

For time-sequential Monte Carlo estimate, it can be used to deal with system with arbitrary dimensions and we can analyse any dynamic system properties by this method, e.g. LOLF or Expected CI. While the simulations are much slower than those by snapshot method, since we

Page: 15

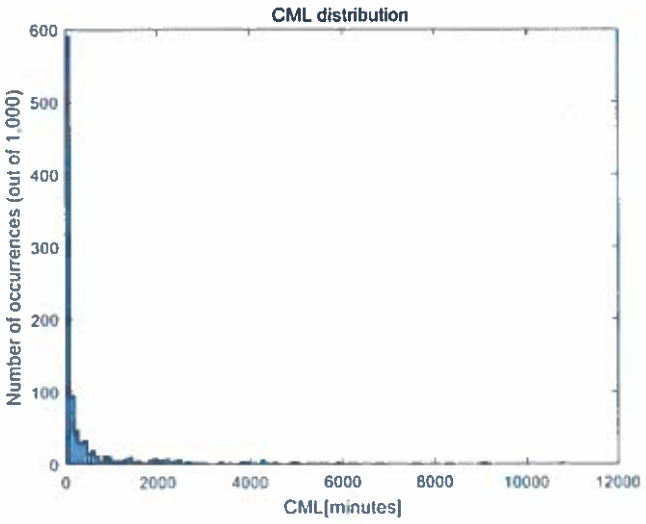
Number: 1 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: as above, express 'close' in terms of the standard error.

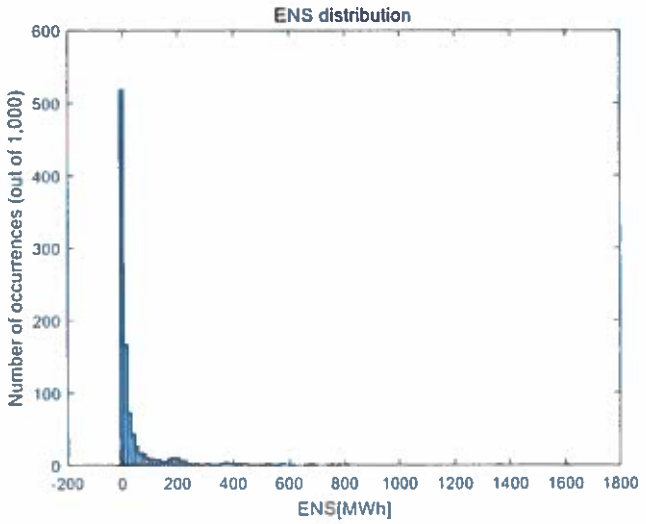
ok

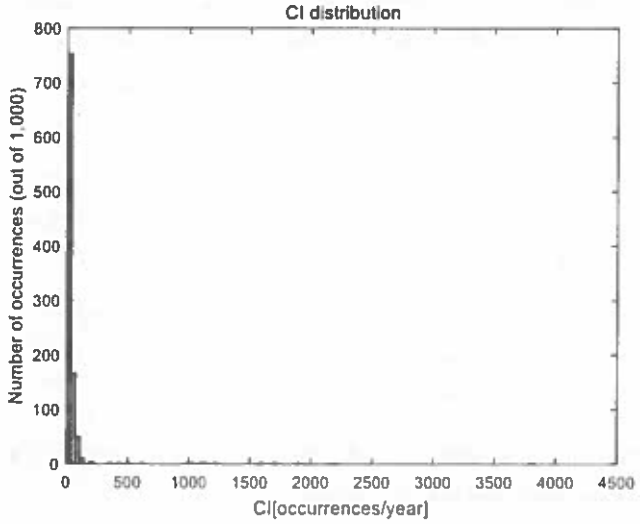
have to generate a long enough path for each sample to ensure Δt to be small enough so that we can have a good insight of the system dynamics.

15. **Plot histograms of the annual CML, CI and ENS values. What drives the 'extreme years' with very high CML and CI values? What could a DNO do to specifically reduce the impact in those years? Provide more than one option, and discuss what information you would need to decide to perform a cost-benefit assessment of these options and decide on the best action.**

We draw the histograms of CML, CI and ENS values as below and the Matlab code for the script **run2.m** is appended at the end of the document







We can derive the range of CML, ENS and CI values from the above histograms and use it to predict a random future year. The distributions of the values can provide intrinsic insight of the system. The figures show the probability to encounter a year with excessive risk metrics. These 'extreme years' might cause regulatory actions and may be unacceptable.

What drives the 'extreme years' with very high CML and CI values?

There are several reasons driving the 'extreme years':

(1) The failure rates of the components are relatively high and the restoration rates of the components are relatively low. That means, once a component turns into a 'Failed State', it might take a relatively long time to back to the 'OK State' and the restoration periods can be very long according to the stochastic processes. 'Extreme years' can be caused by long failure periods of the components

(2) The demand side is not smart. That means, if the loads connected by the feeders excess the available capacity of the transformers, the transformers will trip and all the customers will suffer the power outage. There are no smart grid technologies applied to supply some users, e.g. load shedding or demand response.

(3) The topology of the case system is too simple. That means, once the outage occurs, the only thing we can do is to wait passively the components to restore. If the topology of the system is more complicated, we can take proactive actions to close some other switches to restore the power supply.

ok

What could a DNO do to specifically reduce the impact in those years?

There are several measures that DNO can take to specifically reduce the extreme impacts:

(1) DNO can purchase more assets and provide redundancy for the components so that the availabilities of the network can be enhanced.

(2) DNO can install intelligent components and apply smart grid technologies. In this case, if overloaded, we can shed the excessive load instantly and guarantee the rest customers not to be affected. Also, we prefer load curtailment to be continuous so that the system will be more flexible and reliable[3].

(3) If possible, DNO can incorporate some distribution networks to make the topology of the system more complicated, so that when an outage occurs, we can operate the switches to reroute the possible transmission path.

(4) DNO can degrade the loading levels of the components, in another word, DNO can take more or even full utilizations of the components at peak hours to avoid overload trip. However, this may cause potential risks.

What information you would need to decide to perform a cost-benefit assessment of these options and decide on the best action?

There are some more information required to make beneficial decisions:

(1) The decreased costumer interruption cost as a part of benefit (In UK context, we use customer willing-to-pay to represent the cost associated with disconnections.)

(2) The decreased penalty for poor risk metrics charged by the regulator as a part of benefit

(3) The component cost of the system in order to invest more assets

(4) The lifespan of the components in order to assess long-term cost

(5) The cost of intelligent components to manage load shedding when overloaded

(6) The investing and maintaining cost of incorporating adjacent distribution networks

(7) The cost of repairing and maintaining components in order to measure the potential cost of degrading loading level (increasing utilization) of components

According to the information above, we can compute the cost (including investment, repairing and potential maintenance) of each option that DNO can take. Comparing the costs of these options with the benefits (decreased costumer interruption cost and outage penalty) and we can make the most cost-benefit reasonable action.

ok

Page: 18

Number: 1 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: actually, the VoLL figure is based on WTA

Appendix

1. Part 2-6 sample1.m

```
function [ sampled_state ] = sample1()

load_profile = [0.30 0.25 0.23 0.22 0.21 0.25 0.38 0.52 0.53 0.48
0.45 ...
    0.46 0.49 0.44 0.43 0.48 0.65 0.83 0.86 0.81 0.75 0.70 0.60
0.43];

% samples a random state of the components
T1 = rand < 0.999726;
T2 = rand < 0.999726;
F1 = rand < 0.999452;
F2 = rand < 0.999179;
F3 = rand < 0.999316;
% samples a random time and determine the power consumption per
household
time = randi(24);
load_household = load_profile(time);
% samples the load according to the status of the feeders
load =
F1*load_household*5000+F2*load_household*4000+F2*F3*load_household*20
00;

% return the sampled state as an array with the elements
sampled_state = [T1 T2 F1 F2 F3 load_household load];

end
```

ok

2. Part 2-7 impact1.m

```
function [ LOLP, CML, ENS ] = impact1( state )

% record the status of the components
T1 = state(1);
T2 = state(2);
F1 = state(3);
F2 = state(4);
F3 = state(5);
% record current power consumption per household
load_household = state(6);
% record current load depending on the status of the feeders
load = state(7);
```

Page: 19

Number: 1 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: I would consider this part of the impact calculation.

```
transformer_capacity = (T1+T2)*8000;
consumer_connected = load/load_household;   % number of consumers
connected by the feeders

% transformer overloaded or all the transformers are broken, power
outage
if transformer_capacity < load
    LOLP = 1;
    CML = 8760*60;
    ENS = load_household*11000*8760;
% transformers meet the load condition
else
    LOLP = ~(F1 && F2 && F3);   % if all the feeders are ok
    CML = (11000-consumer_connected)/11000*8760*60;
    ENS = (load_household*11000-load)*8760;
end

end
```

ok

3. Part 2-8 run1.m

```
clear;
SAMPLE_SIZE = 100000;
% initialise empty arrays
LOLP_values = zeros(1, SAMPLE_SIZE);
CML_values = zeros(1, SAMPLE_SIZE);
ENS_values = zeros(1, SAMPLE_SIZE);

for i=1:SAMPLE_SIZE
% generate samples and compute three measurements each time
    state = sample1();
    [LOLP_values(i), CML_values(i), ENS_values(i) ] = impact1(state);
end
ENS_values = ENS_values/1000;  % change the unit from kWh to MWh

% estimate LOLP
LOLP_estimate = mean(LOLP_values);
LOLP_se = std(LOLP_values)/sqrt(SAMPLE_SIZE);
fprintf('LOLP = %.6f ¡À %.6f\n',LOLP_estimate, LOLP_se);
fprintf('LOLP = [%.6f, %.6f] (95%% confidence)\n',LOLP_estimate -
1.96*LOLP_se, LOLP_estimate + 1.96*LOLP_se);
fprintf('LOLP = %.6f
¡À %.1f %%\n',LOLP_estimate,100*LOLP_se/LOLP_estimate);
```

```
% estimate the expectation of CML
exp_CML_estimate = mean(CML_values);
exp_CML_se = std(CML_values)/sqrt(SAMPLE_SIZE);
fprintf('expected-CML = %.2f ¡À %.2f
minites\n',exp_CML_estimate,exp_CML_se);
fprintf('expected-CML = [%.2f, %.2f] minites (95%%
confidence)\n',exp_CML_estimate - 1.96*exp_CML_se, exp_CML_estimate +
1.96*exp_CML_se);
fprintf('expected-CML = %.2f minites
¡À %.1f %%\n',exp_CML_estimate,100*exp_CML_se/exp_CML_estimate);

% estimate EENS
EENS_estimate = mean(ENS_values);
EENS_se = std(ENS_values)/sqrt(SAMPLE_SIZE);
fprintf('EENS = %.2f ¡À %.2f MWh\n',EENS_estimate,EENS_se);
fprintf('EENS = [%.2f, %.2f] MWh (95%% confidence)\n',EENS_estimate -
1.96*EENS_se, EENS_estimate + 1.96*EENS_se);
fprintf('EENS = %.2f MWh
¡À %.1f %%\n',EENS_estimate,100*EENS_se/EENS_estimate);

% visualise the convergence of the results
% add samples 10 by 10 to accelerate the calculation
% visualise LOLP
LOLP_partial_means = zeros(1, SAMPLE_SIZE/10);
LOLP_partial_ses = zeros(1, SAMPLE_SIZE/10);
for i=1:SAMPLE_SIZE/10
% compute partial means and partial standard errors
LOLP_partial_means(i) = mean(LOLP_values(1:10*i));
LOLP_partial_ses(i) = std(LOLP_values(1:10*i))/sqrt(10*i);
end
% plot the true value, the MC estimate and the 95% confidence bands
figure;
plot(1:10:SAMPLE_SIZE, LOLP_partial_means);
hold on;
plot(1:10:SAMPLE_SIZE, LOLP_partial_means + 1.96 * LOLP_partial_ses);
plot(1:10:SAMPLE_SIZE, LOLP_partial_means - 1.96 * LOLP_partial_ses);
plot([1 SAMPLE_SIZE] , [0.002143 0.002143]);
xlabel('samples');
ylabel('LOLP estimate');
legend('MC estimate','upper 95% confidence band','lower 95%
confidence band','analytical value');

% visualise CML
CML_partial_means = zeros(1, SAMPLE_SIZE/10);
```

```
CML_partial_ses = zeros(1, SAMPLE_SIZE/10);
for i=1:SAMPLE_SIZE/10
% compute partial means and partial standard errors
CML_partial_means(i) = mean(CML_values(1:10*i));
CML_partial_ses(i) = std(CML_values(1:10*i))/sqrt(10*i);
end
% plot the true value, the MC estimate and the 95% confidence bands
figure;
plot(1:10:SAMPLE_SIZE, CML_partial_means);
hold on;
plot(1:10:SAMPLE_SIZE, CML_partial_means + 1.96 * CML_partial_ses);
plot(1:10:SAMPLE_SIZE, CML_partial_means - 1.96 * CML_partial_ses);
plot([1 SAMPLE_SIZE] , [479.68 479.68]);
xlabel('samples');
ylabel('CML estimate');
legend('MC estimate','upper 95% confidence band','lower 95%
confidence band','analytical value');

% visualise EENS
EENS_partial_means = zeros(1, SAMPLE_SIZE/10);
EENS_partial_ses = zeros(1, SAMPLE_SIZE/10);
for i=1:SAMPLE_SIZE/10
% compute partial means and partial standard errors
EENS_partial_means(i) = mean(ENS_values(1:10*i));
EENS_partial_ses(i) = std(ENS_values(1:10*i))/sqrt(10*i);
end
% plot the true value, the MC estimate and the 95% confidence bands
figure;
plot(1:10:SAMPLE_SIZE, EENS_partial_means);
hold on;
plot(1:10:SAMPLE_SIZE, EENS_partial_means + 1.96 * EENS_partial_ses);
plot(1:10:SAMPLE_SIZE, EENS_partial_means - 1.96 * EENS_partial_ses);
plot([1 SAMPLE_SIZE] , [45.88 45.88]);
xlabel('samples');
ylabel('EENS estimate');
legend('MC estimate','upper 95% confidence band','lower 95%
confidence band','analytical value');
```

ok

4. Part 3-10 sample2.m

```
function [ state ] = sample2()

load_profile = [0.30 0.25 0.23 0.22 0.21 0.25 0.38 0.52 0.53 0.48
0.45 ...
    0.46 0.49 0.44 0.43 0.48 0.65 0.83 0.86 0.81 0.75 0.70 0.60
```

```
0.43];

% samples a random state of the components
T1 = rand < 0.999726;
T2 = rand < 0.999726;
F1 = rand < 0.999452;
F2 = rand < 0.999179;
F3 = rand < 0.999316;
% initialize time at hour 1
time = 1 ;
% compute the initial power consumption per household
load_household = load_profile(time);
% samples the load according to the status of the feeders
load =
F1*load_household*5000+F2*load_household*4000+F2*F3*load_household*20
00;

% return the sampled state as an array with the elements
state = [T1 T2 F1 F2 F3 load_household load time];
end
```

ok

5. Part 3-11 update_components.m

```
function [ new_state ] = update_components( state )
% update component states after one time step dt
dt = 1;  % delta t = 1 hour

% define the failure and repair rates (in units of 1/hour)
% component order: T1 T2 F1 F2 F3
rates_fail = [1/100 1/100 1/10 1/(20/3) 1/(5/3)]/365/24;  % change
unit to per hour
rates_repair = [1/10 1/10 1/2 1/2 1/(10/24)]/24;  % change unit to
per hour

% update components that change
new_state = state;   % initialize new_state
% Transformer 1
if state(1)==1 % transformer is up
    if rand() < rates_fail(1)*dt
    new_state(1) = 0;
    end
else % transformer is down
    if rand() < rates_repair(1)*dt
    new_state(1) = 1;
    end
```

Number: 1 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: I don't see the update of load levels in this function

```
end

% Transformer 2
if state(2)==1 % transformer is up
    if rand() < rates_fail(2)*dt
    new_state(2) = 0;
    end
else % transformer is down
    if rand() < rates_repair(2)*dt
    new_state(2) = 1;
    end
end

% Feeder 1
if state(3)==1 % feeder is available
    if rand() < rates_fail(3)*dt
    new_state(3) = 0;
    end
else % feeder is unavailable
    if rand() < rates_repair(3)*dt
    new_state(3) = 1;
    end
end

% Feeder 2
if state(4)==1 % feeder is available
    if rand() < rates_fail(4)*dt
    new_state(4) = 0;
    end
else % feeder is unavailable
    if rand() < rates_repair(4)*dt
    new_state(4) = 1;
    end
end

% Feeder 3
if state(5)==1 % feeder is available
    if rand() < rates_fail(5)*dt
    new_state(5) = 0;
    end
else % feeder is unavailable
    if rand() < rates_repair(5)*dt
    new_state(5) = 1;
    end
```

```
end

end
```

6. Part 3-12 next_time_step.m

```
function [ new_state ] = next_time_step( state )
% update whole state of the system after one time step

load_profile = [0.30 0.25 0.23 0.22 0.21 0.25 0.38 0.52 0.53 0.48
0.45 ...
    0.46 0.49 0.44 0.43 0.48 0.65 0.83 0.86 0.81 0.75 0.70 0.60
0.43];
new_state = state;   % initialize new_state
new_state = update_components( state );   % update the state of
components

% update time
if state(8)<24
    new_state(8) = state(8)+1;
else
    new_state(8) = 1;
end
% update household power consumption
load_household = load_profile(new_state(8));
new_state(6) = load_household;
F1 = new_state(3);
F2 = new_state(4);
F3 = new_state(5);
% update load level according to new status of the feeders
new_state(7) =
F1*load_household*5000+F2*load_household*4000+F2*F3*load_household*20
00;

end
```

7. Part 3-13 year_path.m

```
function [LOLP, CML, ENS, CI] = year_path()
T = 24*365;   % number of hours in one year

% initialise impact metrics
LOLP = 0;
CML = 0;
```

Page: 25

Number: 1 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: ah, here it is.

```
ENS = 0;
CI = 0;
% initialise the states of load points 1 = supplied  0 = outage
old_A = 1; old_B = 1; old_C = 1;
A = 1; B = 1; C = 1;

state = sample2();   % initialise the beginning state at hour 1
% old_state = state;
for i=1:T
    if i>1
%         old_state = state;
        state = next_time_step( state );  % change state to the next
time step
    end
    % record the status of the components
    T1 = state(1);
    T2 = state(2);
    F1 = state(3);
    F2 = state(4);
    F3 = state(5);
    % record current power consumption per household
    load_household = state(6);
    % record current load depending on the status of the feeders
    load = state(7);

    transformer_capacity = (T1+T2)*8000;
    consumer_connected = load/load_household;   % number of consumers
connected by the feeders

    % transformer overloaded or all the transformers are broken,
power outage
    if transformer_capacity < load
        d_LOLP = 1;
        d_CML = 1*60;
        d_ENS = load_household*11000*1;
        A = 0; B = 0; C = 0;   % update the current states of load
points
    % transformers meet the load condition
    else
        d_LOLP = ~(F1 && F2 && F3);  % if all the feeders are ok
        d_CML = (11000-consumer_connected)/11000*60;
        d_ENS = (load_household*11000-load)*1;
        A = F1; B= F2; C= F2*F3;   % update the current states of load
points
```

```
    end
    % compare current states of load points with old states and
compute
    % delta CI (only old state is ok and current state is fail
counts)
    d_CI =
(old_A*~(A)*5000+old_B*~(B)*4000+old_C*~(C)*2000)/11000*100;
    % update the old states of load points
    old_A = A; old_B = B; old_C = C;

    %  add delta h of each step and calculate the impact metrics
    LOLP = LOLP + d_LOLP/T;
    CML = CML + d_CML;
    ENS = ENS + d_ENS;
    CI = CI + d_CI;
end

end
```

8. Part 3-14, 3-15 run2.m

```
clear;
SAMPLE_SIZE = 1000;
% initialise empty arrays
LOLP_values = zeros(1, SAMPLE_SIZE);
CML_values = zeros(1, SAMPLE_SIZE);
ENS_values = zeros(1, SAMPLE_SIZE);
CI_values = zeros(1, SAMPLE_SIZE);

for i=1:SAMPLE_SIZE
    % generate samples and compute four impact metrics each time
    [LOLP_values(i), CML_values(i), ENS_values(i), CI_values(i)] =
year_path();
end
ENS_values = ENS_values/1000;  % change the unit from kWh to MWh

% estimate LOLP
LOLP_estimate = mean(LOLP_values);
LOLP_se = std(LOLP_values)/sqrt(SAMPLE_SIZE);
fprintf('LOLP = %.6f ¡À %.6f\n',LOLP_estimate, LOLP_se);
fprintf('LOLP = [%.6f, %.6f] (95%% confidence)\n',LOLP_estimate -
1.96*LOLP_se, LOLP_estimate + 1.96*LOLP_se);
fprintf('LOLP = %.6f
¡À %.1f %%\n',LOLP_estimate,100*LOLP_se/LOLP_estimate);
```

Page: 27

Number: 1 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: nice compact notation

```
% estimate the expectation of CML
exp_CML_estimate = mean(CML_values);
exp_CML_se = std(CML_values)/sqrt(SAMPLE_SIZE);
fprintf('expected-CML = %.2f ¡À %.2f
minites\n',exp_CML_estimate,exp_CML_se);
fprintf('expected-CML = [%.2f, %.2f] minites (95%%
confidence)\n',exp_CML_estimate - 1.96*exp_CML_se, exp_CML_estimate +
1.96*exp_CML_se);
fprintf('expected-CML = %.2f minites
¡À %.1f %%\n',exp_CML_estimate,100*exp_CML_se/exp_CML_estimate);

% estimate EENS
EENS_estimate = mean(ENS_values);
EENS_se = std(ENS_values)/sqrt(SAMPLE_SIZE);
fprintf('EENS = %.2f ¡À %.2f MWh\n',EENS_estimate,EENS_se);
fprintf('EENS = [%.2f, %.2f] MWh (95%% confidence)\n',EENS_estimate -
1.96*EENS_se, EENS_estimate + 1.96*EENS_se);
fprintf('EENS = %.2f MWh
¡À %.1f %%\n',EENS_estimate,100*EENS_se/EENS_estimate);

% estimate the expectation of CI
exp_CI_estimate = mean(CI_values);
exp_CI_se = std(CI_values)/sqrt(SAMPLE_SIZE);
fprintf('expected-CI = %.2f ¡À %.2f
occurrences/year\n',exp_CI_estimate,exp_CI_se);
fprintf('expected-CI = [%.2f, %.2f] occurrences/year (95%%
confidence)\n',exp_CI_estimate - 1.96*exp_CI_se, exp_CI_estimate +
1.96*exp_CI_se);
fprintf('expected-CI = %.2f occurrences/year
¡À %.1f %%\n',exp_CI_estimate,100*exp_CI_se/exp_CI_estimate);

% visualise the convergence of the results
% visualise LOLP
LOLP_partial_means = zeros(1, SAMPLE_SIZE);
LOLP_partial_ses = zeros(1, SAMPLE_SIZE);
for i=1:SAMPLE_SIZE
% compute partial means and partial standard errors
LOLP_partial_means(i) = mean(LOLP_values(1:i));
LOLP_partial_ses(i) = std(LOLP_values(1:i))/sqrt(i);
end
% plot the true value, the MC estimate and the 95% confidence bands
figure;
plot(1:SAMPLE_SIZE, LOLP_partial_means);
hold on;
```

```
plot(1:SAMPLE_SIZE, LOLP_partial_means + 1.96 * LOLP_partial_ses);
plot(1:SAMPLE_SIZE, LOLP_partial_means - 1.96 * LOLP_partial_ses);
plot([1 SAMPLE_SIZE] , [0.002143 0.002143]);
xlabel('years');
ylabel('LOLP estimate');
legend('MC estimate','upper 95% confidence band','lower 95%
confidence band','analytical value');

% visualise CML
CML_partial_means = zeros(1, SAMPLE_SIZE);
CML_partial_ses = zeros(1, SAMPLE_SIZE);
for i=1:SAMPLE_SIZE
% compute partial means and partial standard errors
CML_partial_means(i) = mean(CML_values(1:i));
CML_partial_ses(i) = std(CML_values(1:i))/sqrt(i);
end
% plot the true value, the MC estimate and the 95% confidence bands
figure;
plot(1:SAMPLE_SIZE, CML_partial_means);
hold on;
plot(1:SAMPLE_SIZE, CML_partial_means + 1.96 * CML_partial_ses);
plot(1:SAMPLE_SIZE, CML_partial_means - 1.96 * CML_partial_ses);
plot([1 SAMPLE_SIZE] , [479.68 479.68]);
xlabel('years');
ylabel('expected CML estimate');
legend('MC estimate','upper 95% confidence band','lower 95%
confidence band','analytical value');

% visualise EENS
EENS_partial_means = zeros(1, SAMPLE_SIZE);
EENS_partial_ses = zeros(1, SAMPLE_SIZE);
for i=1:SAMPLE_SIZE
% compute partial means and partial standard errors
EENS_partial_means(i) = mean(ENS_values(1:i));
EENS_partial_ses(i) = std(ENS_values(1:i))/sqrt(i);
end
% plot the true value, the MC estimate and the 95% confidence bands
figure;
plot(1:SAMPLE_SIZE, EENS_partial_means);
hold on;
plot(1:SAMPLE_SIZE, EENS_partial_means + 1.96 * EENS_partial_ses);
plot(1:SAMPLE_SIZE, EENS_partial_means - 1.96 * EENS_partial_ses);
plot([1 SAMPLE_SIZE] , [45.88 45.88]);
xlabel('years');
```

```
ylabel('EENS estimate');
legend('MC estimate','upper 95% confidence band','lower 95%
confidence band','analytical value');

% visualise CI
CI_partial_means = zeros(1, SAMPLE_SIZE);
CI_partial_ses = zeros(1, SAMPLE_SIZE);
for i=1:SAMPLE_SIZE
% compute partial means and partial standard errors
CI_partial_means(i) = mean(CI_values(1:i));
CI_partial_ses(i) = std(CI_values(1:i))/sqrt(i);
end
% plot the true value, the MC estimate and the 95% confidence bands
figure;
plot(1:SAMPLE_SIZE, CI_partial_means);
hold on;
plot(1:SAMPLE_SIZE, CI_partial_means + 1.96 * CI_partial_ses);
plot(1:SAMPLE_SIZE, CI_partial_means - 1.96 * CI_partial_ses);
xlabel('years');
ylabel('expected CI estimate');
legend('MC estimate','upper 95% confidence band','lower 95%
confidence band');

% print histograms of CML, CI and ENS
figure;
histogram(CML_values,120);
title('CML distribution');
xlabel('CML[minutes]');
ylabel('Number of occurrences (out of 1,000)');
figure;
histogram(ENS_values,120);
title('ENS distribution');
xlabel('ENS[MWh]');
ylabel('Number of occurrences (out of 1,000)');
figure;
histogram(CI_values,120);
title('CI distribution');
xlabel('CI[occurrences/year]');
ylabel('Number of occurrences (out of 1,000)');
```

ok

Reference

[1] Simon Tindemans, Lecture Slides of Smart Grid Technologies

[2] Simon Tindemans, Exercise solutions of Smart Grid Technologies

[3] Yang, Yang, Simon Tindemans, and Goran Strbac. "An implicit switching model for distribution network reliability assessment." *Power Systems Computation Conference (PSCC), 2016*. IEEE, 2016.